Trabalho de Otimização Linear

INSTITUIÇÃO IMPA Tech DATA 3 de Outubro de 2026 AUTORES Huann Vicente, Nicole Freire

PARTE 1

Leitura e processamento da imagem

Nessa seção, realizamos a transformação da imagem original em uma imagem binária, bem como identificamos a entrada e a saída do labirinto.

Transformando a imagem na matriz

Para transformação da imagem original na imagem binária, utlizamos a matriz fornecida pela atividade:

$$B(i,j) = \begin{cases} 1, & \text{se o pixel pertence a uma região livre,} \\ 0, & \text{se o pixel pertence a uma parede.} \end{cases}$$

E o código:

MATLAB COPIAR

```
function B = carregar_imagem_binaria(caminho_arquivo)
  pkg load image

  if nargin < 1 || isempty(caminho_arquivo)
    caminho_arquivo = "labirinto.jpeg";
  endif

  img = imread(caminho_arquivo);

  if size(img, 3) == 3
    img = rgb2gray(img);
  endif

```

```
    limiar = 128;

    B_temp = img > limiar;

    [linhas_pretas, colunas_pretas] = find(B_temp == 0);

    y_min = min(linhas_pretas);
    x_min = min(colunas_pretas);
    y_max = max(linhas_pretas);
    x_max = max(colunas_pretas);

    B = B_temp(y_min:y_max, x_min:x_max);

    imwrite(B, "labirinto_41x41.png");
end
```

CÓDIGO 1

Como pode ser visto, a leitura da imagem foi realizada por meio da função `imread`. Caso ela viesse em `RGB`, era convertida para escala de cinza utilizando a função `rgb2gray`. Utilizamos um limiar de 128 pois ele representa o ponto médio da escala cinza de $8\ bits$, a qual varia de 0 = preto absoluto a 255 = branco absoluto (ponto médio = $\frac{0+255}{2} = 127.5 \approx 128$). Como há um grande contraste na imagem (paredes pretas e corredores brancos), o limiar realiza a binarização de forma satisfatória. Obtemos, então, a nossa matriz B de binarização:

$$B(i,j) = \begin{cases} 1, & \text{se } I(i,j) > 128 \\ 0, & \text{caso contrário} \end{cases}$$

Nesta matriz, o valor 1 representa um pixel de corredor, o que equivale a cor branca. Já o valor 0 representa um pixel de parede, o que equivale a cor preta. Além disso, após *croppar* a imagem, identificamos a entrada como o primeiro bloco branco da borda esquerda do labirinto e a saída como primeiro bloco branco da borda direita do labirinto (durante o desenvolvimento do trabalho, utilizaremos uma 💡 função para identificá-los com `id` e posição em coordenadas).

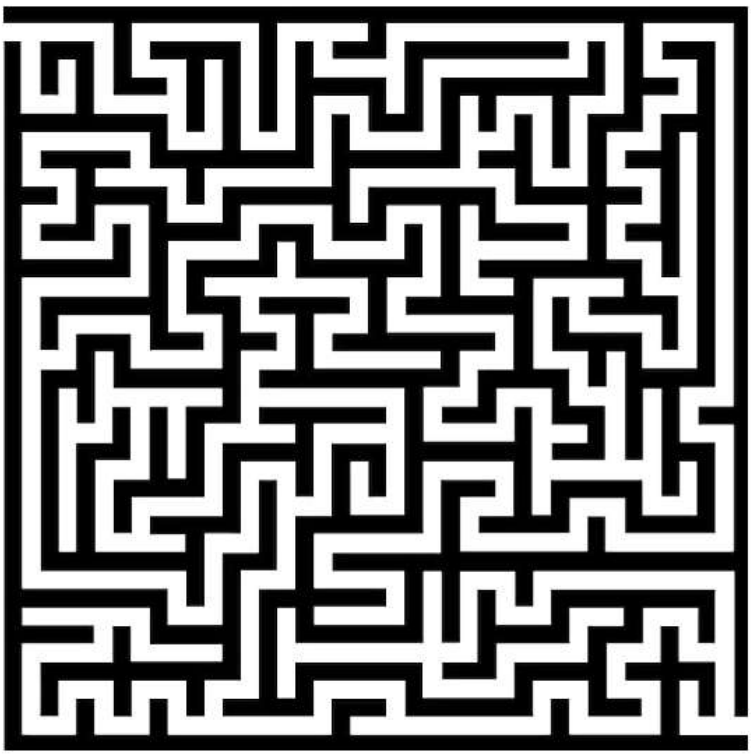


FIGURA 1 Imagem original do labirinto

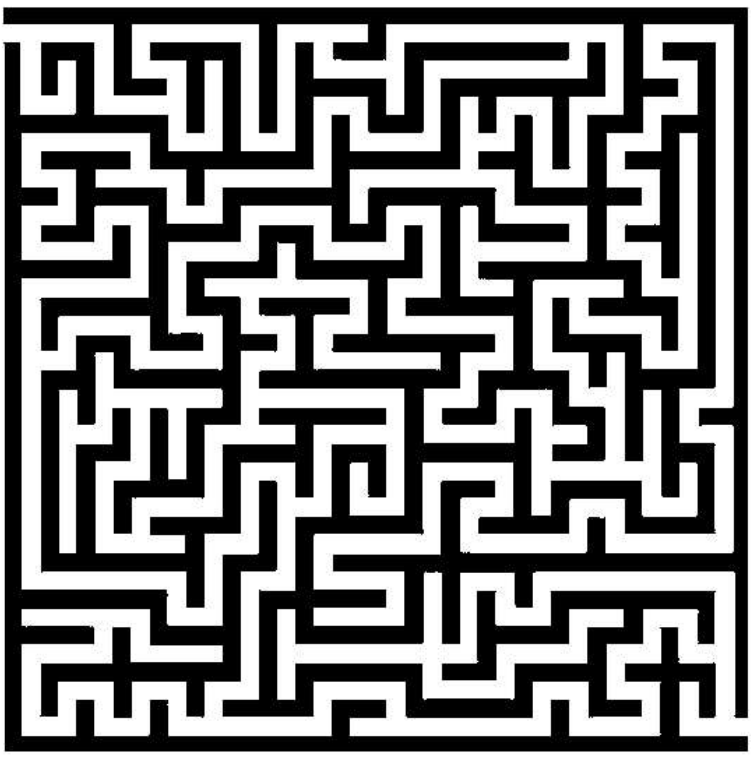


FIGURA 2 Imagem binária obtida do labirinto

PARTE 2

Criando o grafo a partir da imagem

Identificador de uma célula

Utilizamos para criar o **identificador do nó**

$$id(i, j) = (i - 1)n + j \tag{1}$$

Pelo fato da função id ser injetiva pela Proposição 1

– PROPOSIÇÃO 1 (INJETIVIDADE DO IDENTIFICADOR DE CADA NÓ)

Dada a função de identificação de um nó $f : I \times J \to \mathbb{N}$ definida por

$$id(i, j) = (i - 1)n + j \tag{2}$$

Onde $n \in \mathbb{N}$ é a dimensão da matriz e $1 \leq j_1, j_2 \leq n$. Temos que id é uma **função injetiva**.

+ *Demonstração.*

A função foi implementada no módulo `calcular_id_no.m`

MATLAB COPIAR

```
function id = calcular_id_no(n, i, j)
  id = (i - 1) * n + j;
end
```

CÓDIGO 3

Calculando o centro de cada célula

Obtemos a exata posição do centro da célula (i, j) calculando:

a. A posição do centro relativa
b. A posição absoluta da célula

Então calculamos primeiro os pixels médios de uma célula.

> **O centro não pode ser exato**
>
> Note que tanto Δx e Δy são valores retirados pela média considerandos que os pixels estão uniformemente espaçados, mas isso não impede de termos valores decimais que não podem ser representados por pixels.

Como cada célula ocupa cerca de $\Delta x, \Delta y \approx 18$ px o seu centro terá a posição relativa $\frac{\Delta x}{2}$, $\frac{\Delta y}{2}$ em relação a tela. Para determinar sua posição absoluta em relação a imagem, temos que contar a posição da (i, j)-célula, que será equivalente a quantidade de pixels a sua esquerda e acima, dada por $(i-1) \times \Delta x$ e $(j-1) \times \Delta y$ e somar a posição relativa centro, resultando nas seguintes expressões para calcular o centro da célula:

$$\begin{aligned}(i-1) \cdot \Delta x + \Delta x/2 &= i \cdot \Delta x - \Delta x/2 \\ &\Rightarrow X(i) = \lfloor i \cdot \Delta x - \Delta x/2 \rfloor\end{aligned} \tag{7}$$

De forma análoga a Equação 7, temos que

$$Y(j) = \lfloor j \cdot \Delta y - \Delta y/2 \rfloor \tag{8}$$

Onde $X(i) \in \mathbb{N}$ e $Y(i) \in \mathbb{N}$. A partir das relações Equações 7 e Equações 8 implementamos no módulo `calcular_centro_celula.m`

MATLAB COPIAR

```
function [x_centro, y_centro] = calcular_centro_celula(B, i, j, n_celulas)
  if nargin < 4 || isempty(n_celulas)
    n_celulas = 41;
  end

  n = n_celulas;

  [altura_imagem, comprimento_imagem, ~] = size(B);

  delta_x = comprimento_imagem / n;
  delta_y = altura_imagem / n;

  x_centro = round((j - 0.5) * delta_x);
  y_centro = round((i - 0.5) * delta_y);
end
```

CÓDIGO 4

Obtendo os caminhos livres horizontais

O maior problema que temos para verificar a passagem de um labirinto é lidar com o tamanho da imagem com o tamanho dos blocos do labirinto. Para termos uma métrica de distância entre células adotamos o centro de um bloco\célula através de $X(i)$, $Y(j)$ das Equações 7, Equações 8.

A idéia para verificar se há uma passagem entre pares de células e comparar se os dois centro são caminhos livres. Mas isso pode ocasionar em falsos caminhos livres, então faremos a verificação se todos os pixels entre cada centro são todos pretos

> **Custo computacional da Verificação**
>
> Note que na verificação `all(linhas_pixels == 1)` possui custo da forma $O(\Delta x)$ que é pior que $O(1)$: alternativa que comparamos apenas os centros de cada bloco, mas como Δx na prática é pequeno, temos que $O(\Delta x) \sim O(1)$.

MATLAB COPIAR

```
function tem_passagem = verificar_passagem_horizontal(B, i, j1, j2, n_celulas)
  if nargin < 5 || isempty(n_celulas)
    n_celulas = 41;
  endif

  [x1, y1] = calcular_centro_celula(B, i, j1, n_celulas);
  [x2, ~] = calcular_centro_celula(B, i, j2, n_celulas);

  x_inicio = min(x1, x2);
  x_fim = max(x1, x2);

  linha_pixel = B(y1, x_inicio:x_fim);
  tem_passagem = all(linha_pixel == 1);
end
```

CÓDIGO 5

Obtendo os caminhos livres verticais

De forma semelhante a caminhos livres horizontais implementamos o método `verificar_passagem_vertical.m`, desta vez i é fixo.

MATLAB COPIAR

```
function tem_passagem = verificar_passagem_vertical(B, i1, i2, j, n_celulas)
  if nargin < 5 || isempty(n_celulas)
    n_celulas = 41;
  endif

  [x1, y1] = calcular_centro_celula(B, i1, j, n_celulas);
  [~, y2] = calcular_centro_celula(B, i2, j, n_celulas);

  y_inicio = min(y1, y2);
  y_fim = max(y1, y2);

  coluna_pixel = B(y_inicio:y_fim, x1);
  tem_passagem = all(coluna_pixel == 1);
end
```

CÓDIGO 6

Criando o conjunto de arestas do labirinto

Criamos o conjunto de arestas do labirinto apenas ligando os caminhos livres. As arestas podem ser separadas por dois subconjuntos de caminhos livres sendo eles o da horizontal e vertical do labirinto. Combinando os métodos visto na Subseção e Subseção

`verificar_passagem_horizontal`: Verificará todos os pares de vértices na horizontal

`verificar_passagem_vertical`: Veriicará todos os pares de vértices na vertical

MATLAB COPIAR

```
function [arestas, qtd_arestas] = obter_arestas(B, n_celulas)
  if nargin < 2 || isempty(n_celulas)
    n_celulas = 41;
  end

  n = n_celulas;

  max_arestas = 2 * n^2; % Total de arestas possíveis
  arestas_temp = zeros(max_arestas, 2);
  qtd_arestas = 0;

  for i = 1:n
    for j = 1:n
      if j < n
        if verificar_passagem_horizontal(B, i, j, j + 1, n)
          id_u = calcular_id_no(n, i, j);
          id_v = calcular_id_no(n, i, j + 1);
          qtd_arestas = qtd_arestas + 1;

          arestas_temp(qtd_arestas, :) = [id_u, id_v];
        endif
      endif
      if i < n
        if verificar_passagem_vertical(B, i, i + 1, j, n)
          id_u = calcular_id_no(n, i, j);
          id_v = calcular_id_no(n, i + 1, j);
          qtd_arestas = qtd_arestas + 1;

          arestas_temp(qtd_arestas, :) = [id_u, id_v];
```

```
        endif
      endif
    end
  end

  arestas = arestas_temp(1:qtd_arestas, :);
end
```

CÓDIGO 7

PARTE 3

Visualização do grafo

MATLAB COPIAR

```
function visualizar_grafo(img, arestas, vertices, s, t, n_celulas, caminho)
  if nargin < 7
    caminho = [];
  endif
  if nargin < 8 || isempty(mostrar_grafo)
    mostrar_grafo = true;
  endif
  n = n_celulas;

  % centro = (x = coluna, y = linha) de todas as celulas, indexado pelo id de
  x_c = zeros(n^2, 1);
  y_c = zeros(n^2, 1);
  for i = 1:n
    for j = 1:n
      id = obter_id_no(n, i, j);
      [x_c(id), y_c(id)] = obter_centro_celula(img, i, j, n);
    end
  end

  imshow(img);
  hold on;

  % desenha todas as arestas em uma única chamada utlizando a estratégia dos
  if mostrar_grafo
    u = arestas(:, 1);
    v = arestas(:, 2);
    X = [x_c(u)'; x_c(v)'; NaN(1, numel(u))];
    Y = [y_c(u)'; y_c(v)'; NaN(1, numel(u))];
```

```
29        plot(X(:), Y(:), "-", "Color", [0.90 0.50 0.00], "LineWidth", 1);
30
31      % plota os vértices
32      plot(x_c(vertices), y_c(vertices), "o", "MarkerSize", 3, ...
33            "MarkerFaceColor", [0.25 0.25 0.25], "MarkerEdgeColor", "none");
34      endif
35
36      if ~isempty(caminho)
37        plot(x_c(caminho), y_c(caminho), "-", "Color", [0.85 0.00 0.00], "LineWic
38      endif
39
40      % entrada com cor verde e saída com cor azul, como sugerido na atividade
41      plot(x_c(s), y_c(s), "s", "MarkerSize", 5, "MarkerFaceColor", [0.00 0.65 0.
42      plot(x_c(t), y_c(t), "s", "MarkerSize", 5, "MarkerFaceColor", [0.00 0.20 0.
43
44      hold off;
45    end
```

CÓDIGO 8

Para realizarmos a visualização do grafo, a função `visualizar_grafo` desenha o grafo sobre a própria imagem, utilizando as coordenadas dos centros das células. Os **vértices** são representados por meio de um ponto cinza no centro de cada uma das células livres. As **arestas** são representadas por meio de um segmento laranja entre os centros de cada par de células livres ligadas. Todos os segmentos foram traçados em uma única chamada da função `plot`, por meio da estratégia dos `NaN`. A sua utilização permite renderizar todas as arestas do grafo numa única chamada, eliminando a necessidade de ciclos `for`: como o MATLAB interpreta o `NaN` como um ponto de descontinuidade (interrompendo a linha sem conectar arestas distintas), foi possível organizar as coordenadas num único vetor intercalado ($[X_1, X_2, \mathrm{NaN}, \dots]$). Esta abordagem de vetorização reduz drasticamente o tempo de execução e o custo computacional de renderização, sendo essencial para garantir a eficiência do código em grafos de grande escala. Já em relação à **entrada** e à **saída**, identificamos elas por meio de quadrados maiores, com cor verde e cor azul, respectivamente, como foi sugerido na atividade.

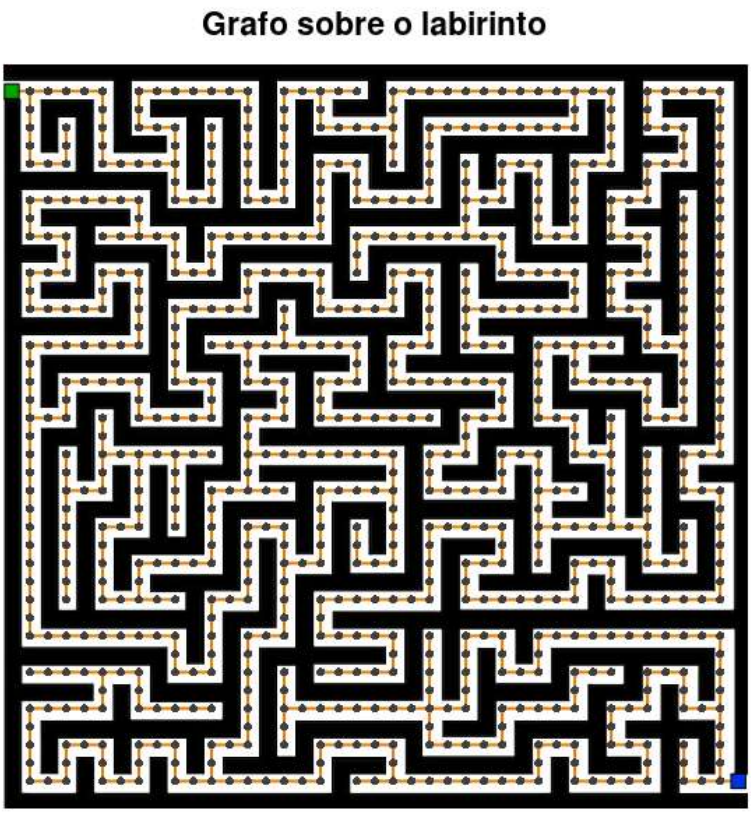


FIGURA 3 Grafo desenhado sobre o labirinto

Podemos verificar visualmente de forma direta que nenhuma aresta cruza uma região preta. Isso já é garantido pela construção que realizamos, em que uma aresta só existe quando todos os pixels do segmento entre os vértices que a compõem são brancos.

PARTE 4

Formulação como problema de caminho mínimo

DEFINIÇÃO 1 (CAMINHO MÍNIMO)

Seja $G = (V, E)$ o grafo do labirinto, s a entrada e t a saída. O **caminho mínimo** entre s e t é o caminho $P = (s, v_1, v_2, ..., v_k, t)$ que minimiza

$$C(P) = \sum_{e \in P} c_e, \qquad c_e = 1 \ \ \forall e \in E$$

O primeiro passo para formularmos nosso problema como um problema de de caminho mínimo é identificamos os vértices s e t, os quais são as entrada e a saída do nosso labirinto, respectivamente. Esse feito é realizado por meio da função `obter_entrada_saída`, a qual nos retorna os parâmetros `s` e `t`, os quais guardam os `ids` da entrada e da saída do nosso labirinto, e os parâmetros `pos_s` e `pos_t`, os quais armazenam a posição desses vértices em coordenadas.

MATLAB COPIAR

```
function [s, t, pos_s, pos_t] = obter_entrada_saida(B, n_celulas)
  if nargin < 2 || isempty(n_celulas)
    n_celulas = 41;
  endif

  n = n_celulas;
  [livres, ~, ~, ~] = obter_info_caminhos_paredes(B, n);

  i = ceil(livres / n);
  j = livres - (i - 1) * n;

  na_borda = (i == 1) | (i == n) | (j == 1) | (j == n);
  ids_borda = livres(na_borda);

  if numel(ids_borda) ~= 2
    error("Esperava-se 2 celulas livres na borda (entrada e saida). Encontrac
  endif

  s = ids_borda(1);   % menor id: entrada
  t = ids_borda(2);   % maior id: saida

  pos_s = [ceil(s / n), s - (ceil(s / n) - 1) * n];   % [linha, coluna]
  pos_t = [ceil(t / n), t - (ceil(t / n) - 1) * n];
end
```

CÓDIGO 9

Ao rodarmos essa função, obtemos que s é a célula $(2, 1)$, de id 42, e t é a célula $(40, 41)$, de id 1640.

De posse desses dados, podemos iniciar a formulação do nosso problema como problema de caminho mínimo. Mas, aqui caberia uma discussão:

Por que podemos formulá-lo como problema de caminho mínimo?
A resposta é simples: Como cada célula livre é um vértice e cada passagem entre células vizinhas é uma aresta, andar pelo labirinto é o mesmo que percorrer arestas. Então, ir da entrada à saída com o menor número de passos é achar o caminho de menor custo entre s e t. Logo, podemos formulá-lo como problema de caminho mínimo.

Além disso, a interpretação de custo para esse problema seria que como todo movimento custa 1, $C(P)$ é o número de arestas do caminho, isto é, o número de movimentos realizados. Um caminho com m vértices (contando s e t) tem custo $m - 1$.

PARTE 5

Algoritmo Dijkstra

O **Algoritmo de Dijkstra** é adota uma estratégia **gulosa** para determinar o caminho mínimo a partir de um único vértice de origem s até todos os outros vértices de um grafo $G = (V, E)$.

O algoritmo inicializa definindo o custo das transições entre células. No nosso labirinto, cada movimento nas direções cardinais $\leftarrow$, $\uparrow$, $\rightarrow$, $\downarrow$ possui custo unitário $c_{uv} = 1$. Atribuímos à origem s a distância inicial $dist(s) = 0$ e a todos os demais vértices $dist(v) = \infty$(como por exemplo na Figura 4).

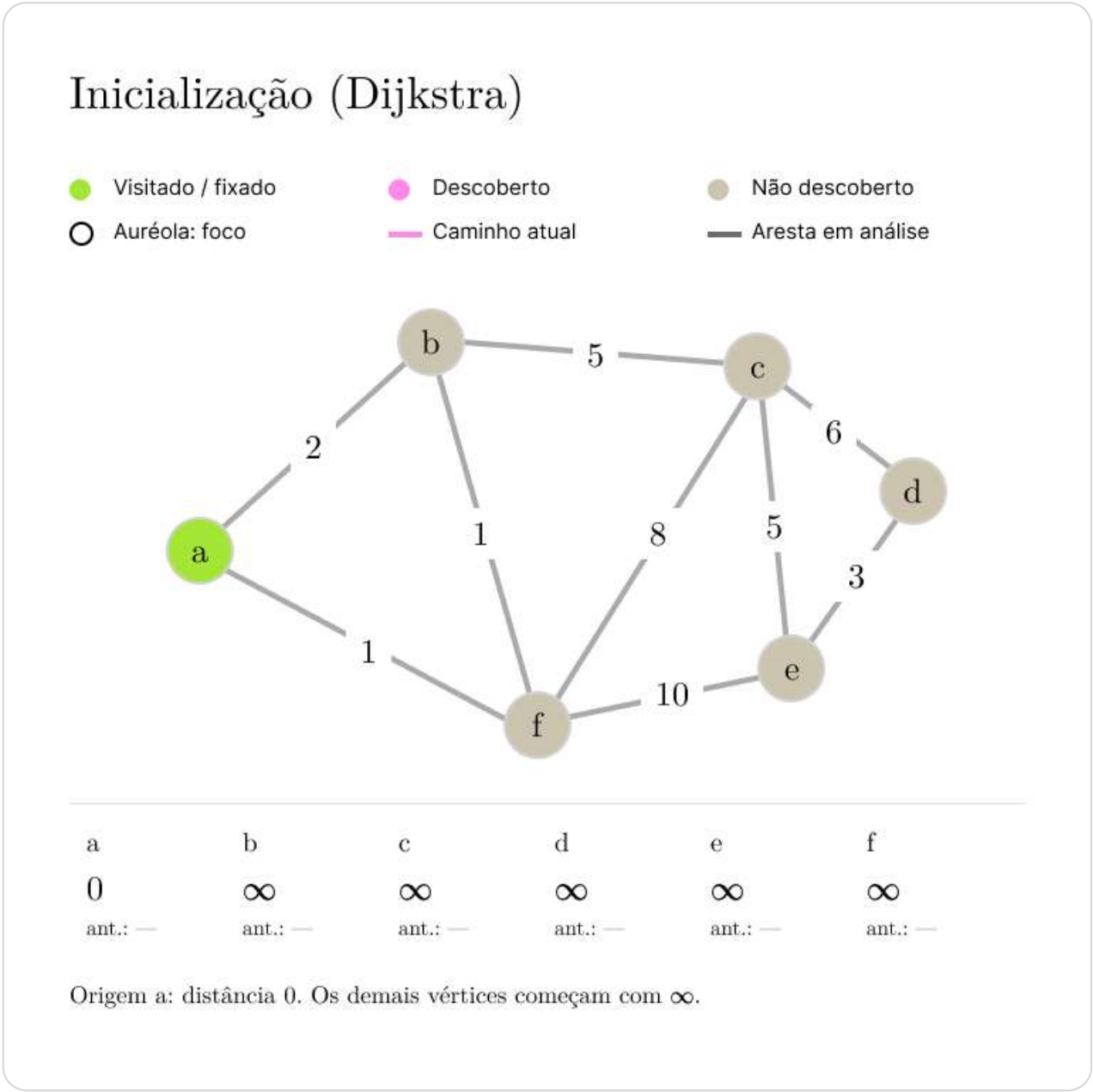


FIGURA 4 Sequência de passos do Algoritmo de Dijkstra ilustrando o processo de seleção de nós, análise de arestas vizinhas e o relaxamento de distâncias.

O coração do algoritmo reside no processo de **relaxamento das arestas**. Para cada vértice u extraído da fila de prioridades, inspecionamos todos os seus vizinhos v acessíveis. A condição:

$$dist(u) + A(u, v) < dist(v) \tag{9}$$

Verifica se a rota que passa por u oferece um caminho mais curto até v do que qualquer outro caminho anteriormente encontrado. Caso essa desigualdade seja satisfeita, atualizamos a menor distância conhecida $dist(v) \leftarrow dist(u) + A(u, v)$

e registramos o predecessor $anterior(v) \leftarrow u$, permitindo a reconstrução exata do trajeto ótimo ao final da execução.

MATLAB COPIAR

```
function [dist, anterior] = dijkstra(A, s)
  N = size(A, 1);

  dist = Inf(N, 1);
  anterior = zeros(N, 1);

  dist(s) = 0;

  heap_distancias = zeros(N * 4, 2);
  heap_tamanho = 0;

  [heap_distancias, heap_tamanho] = heap_push(heap_distancias, heap_tamanho, 0, s);

  while heap_tamanho > 0
    % Extrai o nó de menor distância atual O(log V)
    [heap_distancias, heap_tamanho, d_u, u] = heap_pop(heap_distancias, heap_tamanh

    % Descarta registros desatualizados da heap
    if d_u > dist(u)
      continue;
    endif

    % Relaxamento das arestas dos vizinhos de u
    vizinhos = find(A(u, :));
    for v = vizinhos
      peso = A(u, v);
      nova_dist = dist(u) + peso;

      % Condição de Relaxamento: Encontrou um caminho mais curto até v?
      if nova_dist < dist(v)
        dist(v) = nova_dist;
        anterior(v) = u; % Registra o predecessor para o backtracking

        [heap_distancias, heap_tamanho] = heap_push(heap_distancias, heap_tamanho,
      endif
    endfor
  endwhile
end
```

CÓDIGO 10

Então implementamos o método `dijkstra` que utiliza a estrutura de um Min-Heap para reduzirmos a complexiade com auxilio dos métodos `heap_push` e `heap_pop` (métodos de remoção e inserção de um Heap Binário). Todo o algoritmo pode ser discretizado nas seguintes etapas:

Inicialização das Estruturas: Atribuição das distâncias acumuladas iniciais e inicialização dos seletores em tempo $\mathcal{O}(|V|)$.

Extração do Vértice Mínimo (`heap_pop`): Remoção do elemento de menor distância temporária da fila de prioridades, realizada exatamente uma vez por vértice processado, totalizando $|V|$ execuções.

Relaxamento das Arestas (`heap_push`): Atualização das distâncias estimadas dos vértices adjacentes e subsequente reorganização da estrutura hierárquica, executada no máximo uma vez para cada aresta incidente, totalizando até $|E|$ atualizações.

A Estrutura de Dados Min-Heap Bidimensional

A Min-Heap foi implementada utilizando uma matriz de dimensão $M \times 2$ para atuar como uma **Fila de Prioridades** de alta performance no Octave. A primeira coluna armazena a **distância acumulada** d (utilizada como chave de ordenação para manter a menor distância na raiz), enquanto a segunda coluna armazena o **identificador do vértice** u associado. Essa representação matricial contígua substitui o uso de `structs` pesadas e permite trocas de linhas diretas na memória, garantindo que as operações de inserção (`heap_push`) e remoção do mínimo (`heap_pop`) sejam executadas de forma otimizada em tempo $\mathcal{O}(\log |V|)$.

Reconstrução do caminho e resultados

Para refazermos o caminho, como cada vértice aponta para seu **pai** pela pelo método `anterior`, então implementamos o módulo `reconstruir_caminho.m` para reconstruir qualquer caminho.

MATLAB COPIAR

```
function caminho = reconstruir_caminho(anterior, s, t)
  caminho = [];
  v = t;

  while v ~= 0
    caminho = [v, caminho];

    if v == s
      break;
    endif

    v = anterior(v);
  endwhile

  if isempty(caminho) || caminho(1) ~= s
    caminho = [];
  endif
end
```

CÓDIGO 11

Portanto os resultados obtidos através da implementações `dijkstra` e `reconstruir_caminho` confirmam a eficiência e a exatidão do Algoritmo de Dijkstra integrado à reconstrução do caminho. Para a malha discretizada do labirinto, o trajeto ótimo foi determinado com os seguintes resultados:

Custo do caminho mínimo: 246 unidades;

Movimentos realizados: 246 movimentos ortogonais;

Quantidade de vértices no caminho:: 247 vértices;

Tempo de uma única execução: $\approx 0,106$ segundos;

Tempo médio (20 execuções): $\approx 0,0916$ segundos.

PARTE 7

Caminho mínimo sobre a imagem

Após a realização de todos os algoritmos necessários, foi possível obter as seguintes visualizações:

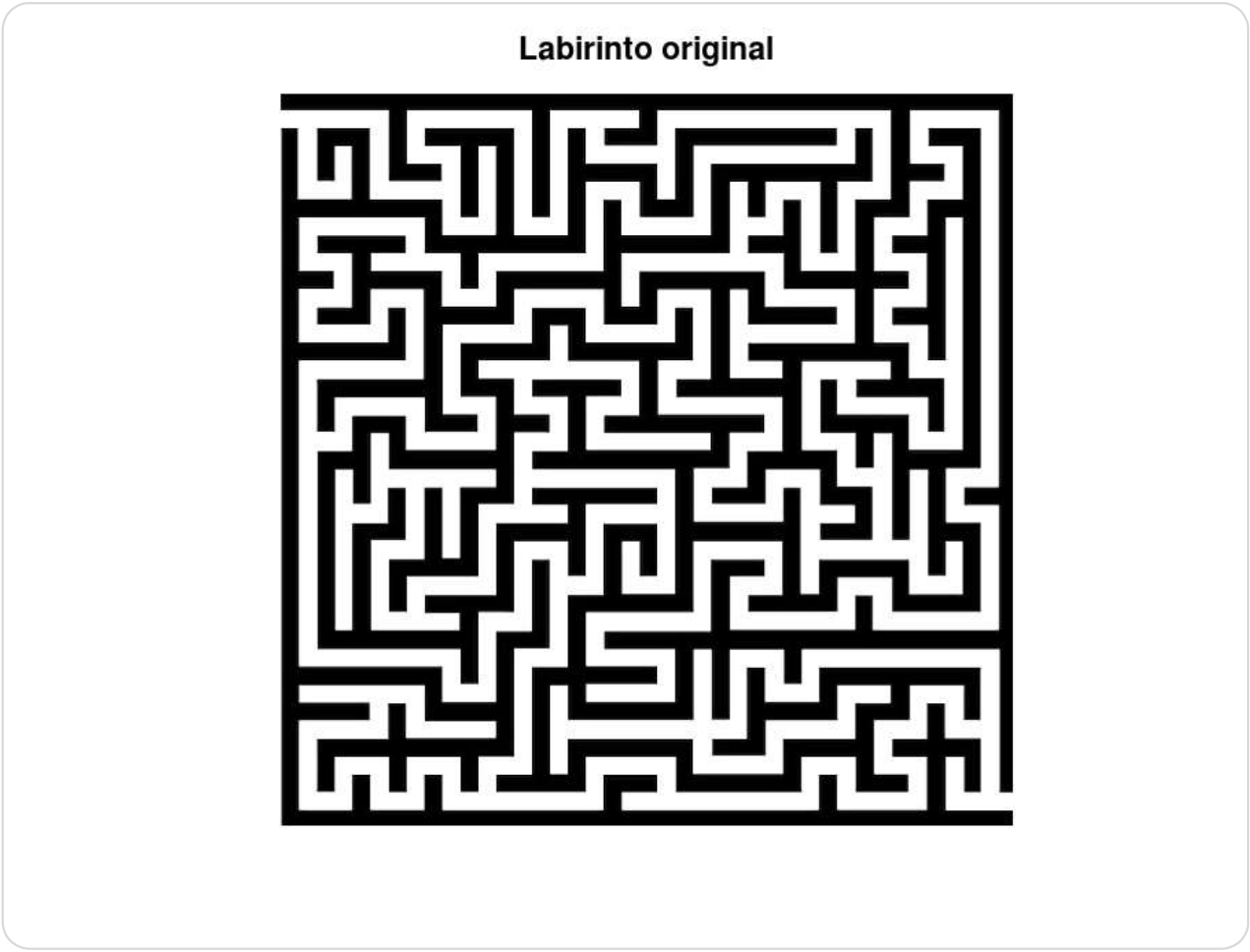


FIGURA 5 Labirinto original



FIGURA 6 Grafo sobre o labirinto

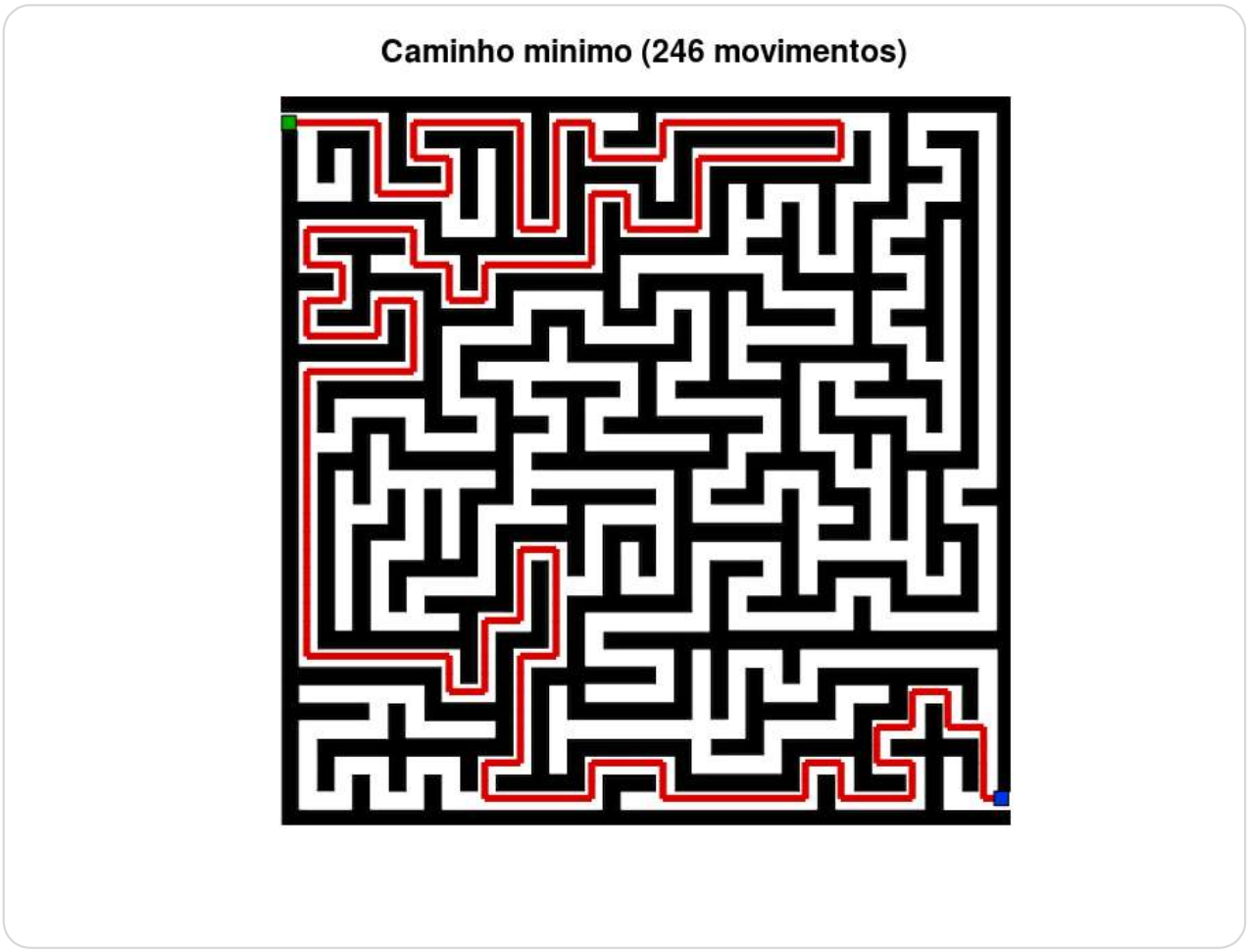


FIGURA 7 Caminho mínimo sobre o labirinto



FIGURA 8 Grafo completo com o caminho mínimo destacado

O grafo tem 801 vértices e 800 arestas, ou seja, uma aresta a menos que vértices. Um grafo conexo com essa propriedade é uma árvore: não tem ciclos, e entre quaisquer dois vértices existe um único caminho. Isso é esperado em um labirinto "perfeito", onde todo corredor é alcançável e não há laços.

As consequências desse fato para o nosso problema são:

> O caminho encontrado não só é o mais curto, mas é o único caminho possível entre s e t, sem repetir vértices;
>
> Qualquer busca (em largura, em profundidade ou Dijkstra) devolveria o mesmo caminho. O Dijkstra não era necessário para este labirinto, mas continua correto e serviria sem alteração se houvesse ciclos ou custos diferentes por aresta;
>
> Os becos sem saída correspondem às folhas da árvore. O algoritmo os explora, mas eles não entram no caminho final.

O resultado final obtido atendeu às expectativas do grupo, gerando o que foi solicitado na atividade.